Plan Offero — Parte 2 de 10

Roles, reparto de módulos y visión de producto · Versión 2 · 30 de octubre de 2026 · Entrega del MVP: viernes 29 de enero de 2027 [SUPUESTO] · Product Owner y Scrum Master: Diego Alejandro Cely Ruiz

> Reparto decidido por el PO: Diego M1 y M4 · Wilson M3 y M5 · Iker M7 y M6 · Daniel M0 y M2. **Cambio de la v2:** Wilson pasa a ser responsable de la base de datos y Daniel coordina la documentación (D-10). Las fechas son las corregidas de la Parte 4.

3. Roles y responsabilidades

3.1 Product Owner y Scrum Master

- **Product Owner (PO):** decide qué se construye y en qué orden. **Diego, sin rotación.** Es quien habla con el profesor y con las empresas. Si el PO cambiara cada sprint, las prioridades cambiarían con él.
- **Scrum Master (SM):** facilita el proceso y elimina bloqueos. Formalmente lo sigue siendo **Diego**, pero **la facilitación de las ceremonias rota**:

Ceremonia	Quién facilita	Por qué
Sprint Planning y Review	Diego	Como PO, define el objetivo y presenta al profesor
Daily asíncrona (resumen y alertas)	Rota cada sprint: Iker → Daniel → Wilson → Iker…	Quita trabajo a Diego y todos aprenden a detectar bloqueos
Retrospectiva	Rota igual, nunca Diego	Como PO también es parte interesada; con alguien neutral salen más fácil los problemas reales

Cómo combinar los roles con programar en un equipo de 4:

- Diego reserva unas 2 h por semana para PO/SM (ya descontadas de la capacidad).
- Se cumple el orden de prioridades de su guía: desbloquear a otros, luego revisar, luego programar lo suyo.
- Los demás no tienen rol de Scrum fijo, sino un rol transversal pequeño (3.2) de alrededor de 1 h por semana.

3.2 Roles transversales

Rol	Titular	Respaldo	Qué hace
Repositorio y revisiones	**Diego**	Iker	Protege main, fusiona con merge commit y aprueba todo PR que toca autenticación o archivos compartidos. Los demás PR los puede aprobar cualquier compañero
Base de datos	**Wilson**	Diego	Cuida el esquema y las convenciones, revisa todo PR con SQL y lo aplica en `offero-prod`. Mantiene la nota «Esquema de la base de datos» en la bóveda. Transición: Sprint 0 y 1 junto con Diego
Seguridad (RLS)	**Iker**	Wilson	Revisa cada tabla y vista con la checklist de RLS e intenta «romper» permisos en las pruebas cruzadas

Rol	Titular	Respaldo	Qué hace
Despliegue	**Diego** (dueño de Vercel)	Daniel	Variables de entorno, URLs de Auth y revisar que `offero-prod` no esté pausado antes de cada demo. Daniel conoce el proceso porque montó M0
Pruebas (QA)	**Daniel**	Wilson	Checklist por módulo, organiza la prueba cruzada al cierre de cada sprint y registra errores en la sección 10
Documentación	**Daniel**	Wilson	Coordina la bóveda de Obsidian (`docs/`) y el contexto del proyecto. Cada autor documenta su módulo; Daniel revisa al cierre de cada sprint que esté hecho

- **Por qué Wilson en base de datos:** tiene un poco más de experiencia que el resto y sabe algo de JavaScript. Aprende PostgreSQL en el Sprint 0 y revisa el SQL junto con Diego en el Sprint 1; desde el Sprint 2 lidera. Así la base de datos deja de depender solo de Diego.
- **Por qué Daniel en documentación:** era el respaldo y es quien más ve el producto completo como QA. Le toma ≈0,5 h por semana, porque cada dueño escribe lo suyo.
- **Respaldo para el «factor bus»:** la base de datos tiene dos personas (Wilson y Diego) y la autenticación también (Diego e Iker). Si alguien falta una semana, su respaldo revisa y aplica.

3.3 Matriz RACI

R = lo hace · **A** = responde por el resultado (uno solo) · **C** = se le consulta · **I** = se le informa

Actividad	Diego	Iker	Daniel	Wilson
Priorizar el Product Backlog	A/R	C	C	C
Sprint Planning	A/R	R	R	R
Facilitar la Daily y la Retro	A	R (rota)	R (rota)	R (rota)
Review y demostración al profesor	A/R	R	R	R
Diseño del esquema y SQL en `offero-prod`	C	C	C	A/R
Escribir las tablas y políticas de cada módulo	A	R	R	R
Revisión de seguridad (RLS)	C	A/R	I	C
Revisar pull requests	A/R	R	R	R
Fusionar a main	A/R	I	I	I
Despliegue y variables en Vercel	A/R	I	C	I
Pruebas cruzadas (QA)	I	R	A/R	R
Bóveda de Obsidian y contexto del proyecto	C	R	A/R	R
Entrevistas a empresas de Bogotá	A/R	C	R	C
Decisiones de producto (sección 11)	A/R	C	C	C
Presentación final (29 ene)	A/R	R	R	R

En la fila «Escribir las tablas y políticas de cada módulo», cada uno hace las de sus propios módulos. Diego también figura con R en sus módulos.

4. Reparto de módulos

Tú, como PO, ya decidiste el reparto. Esta sección lo justifica, lo compara con el del documento y define el trabajo en pareja y la integración de Wilson.

4.1 Reparto del documento frente al decidido

Módulo	Documento de contexto	Decidido	Justificación
M0 Página publicada	Diego	**Daniel**	Complejidad baja: es una buena primera victoria. Libera a Diego para las cuentas y M1. Vercel y Supabase igual los configura Diego
M1 Autenticación y roles	Iker	**Diego**	Es el más difícil y está primero en la ruta crítica. Diego ya trabajó con Supabase Auth
M2 Negocios y categorías	Daniel	**Daniel**	Sin cambio. Encaja con su rol de dueño del diseño base
M3 Solicitudes	Diego	**Wilson**	Complejidad media, muy parecido a su app de tareas en Flask (crear, listar, detalle, estado)
M4 Cotizaciones	Iker	**Diego**	Segundo más difícil, en la ruta crítica, con la operación atómica de aceptar
M5 Mensajería	Daniel	**Wilson**	Se construye sobre las solicitudes, que también son suyas: menos coordinación
M6 Reseñas	Daniel	**Iker**	Con margen de horas y como revisor de seguridad, la regla de «quién puede reseñar» queda en buenas manos
M7 Administración	Diego	**Iker**	Toca todas las tablas: encaja con su rol de revisor de RLS. Ya no está en la ruta crítica

4.2 Carga por persona

Persona	Módulos	Carga estimada	Disponible (Parte 4)	Estado
Diego	M1, M4 + PO/SM + revisiones de auth	≈73 h	≈93 h	Cabe
Wilson	M3, M5 + base de datos	≈66 h (+8 h con tiempo real)	≈76 h	Cabe
Iker	M7, M6 + RLS + «Mis cotizaciones»	≈69 h	≈76 h	Cabe
Daniel	M0, M2 + QA + diseño + documentación	≈72 h	≈76 h	Justo

> **Dos observaciones honestas. Daniel queda justo:** si se atrasa, primero se le quita QA de enero, no la documentación. **Iker pierde los dos módulos que tenía en el documento (M1 y M4):** conviene hablarlo con él directamente, no solo pasarle la guía. Sus módulos y el rol de seguridad tocan todo el sistema, y explicárselo así evita que lo sienta como una degradación.

4.3 Trabajo en pareja

En el trabajo en pareja (*pair programming*), dos personas trabajan juntas en el mismo código: una escribe y la otra revisa y guía.

Qué	Pareja	Cuándo	Para qué
Montar M0	Daniel (escribe) + Wilson (guía)	vie 30 – sáb 31 oct	Wilson aprende la estructura del proyecto desde el primer día
Diseño del esquema de la base de datos	Los 4, facilitan Diego y Wilson (2 h)	jue 5 nov	Wilson empieza a ser dueño del esquema; todos entienden las relaciones
M1: trigger de perfiles, funciones de ayuda y RLS	Diego (escribe) + Iker (guía y pregunta)	lun 2 – sáb 7 nov, 2 sesiones de 1,5 h	Factor bus: Iker queda capaz de mantener la autenticación
Políticas de requests	Wilson (escribe) + Iker (guía)	mar 10 – mié 11 nov	Su primera política RLS; Iker ya practicó en el Sprint 0
Revisión del SQL del Sprint 1	Wilson + Diego	Sprint 1	Transición: Wilson aprende a revisar SQL antes de liderar
Función de aceptar cotización (M4)	Diego (escribe) + Iker (revisa la seguridad)	jue 3 dic, 1 h	Es la función con más privilegios del sistema; la revisan Iker y Wilson
Función `can_message` **(M5)**	Wilson (escribe) + Diego (guía)	dom 13 dic, 1 h	La política RLS más compleja que escribe Wilson

4.4 Plan de integración de Wilson (Sprint 0)

Cuándo	Qué hace	Cómo se verifica
Hoy, jue 29 oct	Acepta las invitaciones de GitHub y de Supabase (la de Supabase vence en 24 h); entra a Discord	Aparece como colaborador y como miembro de la organización
vie 30 oct	Lee completos el documento de contexto y su guía; prepara su computador (Anexo B de la guía)	`node -v`, `git --version` y `npx -v` funcionan
vie 30 – sáb 31 oct	Trabaja en pareja con Daniel en M0	Sabe explicar qué hacen `page.js` y `layout.js`
lun 2 – mar 3 nov	Clona, corre la app y hace su **primer PR**: agrega su fila a la tabla de integrantes	PR revisado y fusionado, sin archivos .rar ni `node_modules`
lun 2 – vie 6 nov	Aprende **PostgreSQL** (tablas, tipos, claves, restricciones, JOIN) practicando en offero-dev, y React y App Router	Crea y consulta una tabla de práctica con clave foránea y CHECK
jue 5 nov	Co-facilita con Diego la sesión de diseño de la base de datos	Explica cómo se relacionan `requests`, `categories` y `profiles`
sáb 7 nov	Revisión de 20 minutos con Diego	Crear una rama, hacer un commit, abrir un PR y actualizar su rama con main, sin ayuda
dom 8 nov	Primera Sprint Planning, donde toma sus historias de M3	Sus tareas están en el tablero con fecha

- **Compañero de referencia:** Daniel. Es a quien Wilson pregunta primero, antes de recurrir a Diego.
- **Base de datos:** en el Sprint 1 revisa el SQL junto con Diego; desde el Sprint 2 revisa y aplica en prod él solo, con Diego de respaldo.

5. Visión de producto y MVP

5.1 Product Goal

El Product Goal es el objetivo de producto de largo plazo que orienta todos los sprints.

> **Que empresas y compradores de Bogotá publiquen lo que necesitan, reciban cotizaciones de proveedores reales y elijan una dentro de Offero, y que para el 29 de enero ambos lados lo hayan usado con transacciones reales.**

5.2 Hipótesis que el MVP debe validar

Las metas son pequeñas a propósito, porque el piloto arranca con «un par de empresas». Son un [SUPUESTO] que Diego ajusta según las entrevistas.

#	Hipótesis	Cómo sabremos si se validó (antes del 29 ene)
H1	Los compradores prefieren publicar una solicitud y recibir propuestas antes que buscar proveedores uno por uno	Al menos 2 compradores del piloto publican 3 o más solicitudes reales en total, y en la entrevista dicen que les ahorró tiempo
H2	Los proveedores responden a solicitudes de desconocidos	Al menos el 60 % de las solicitudes reales reciben 1 o más cotizaciones en 72 h
H3	Comparar en un mismo lugar (precio, plazo, condiciones) facilita decidir	En al menos 2 solicitudes el cliente elige una cotización dentro de la app
H4	Las reseñas generan confianza para contratar a alguien nuevo	Hay al menos 1 reseña real, y en las entrevistas los compradores dicen que la tendrían en cuenta
H5	El modelo de negocio (quién pagaría y cómo)	**El MVP no la valida.** Se explora en las entrevistas (sección 16) porque los pagos están fuera del MVP

Si una hipótesis no se valida, también es un resultado útil para la presentación: muestra que el equipo aprendió algo del mercado y no solo que construyó software.

5.3 Actores y qué necesita cada uno

Actor	Qué necesita
Cliente comprador	Registrarse rápido; describir lo que necesita (producto individual, personalizado o al por mayor); recibir varias propuestas sin perseguir proveedores; compararlas claramente; hablar con el proveedor elegido; saber en quién confiar
Proveedor	Un perfil de negocio que lo muestre; ver solicitudes abiertas de su categoría; cotizar fácil; saber si ganó o perdió; contacto directo con el cliente
Administrador	Mantener las categorías; ver los usuarios; recibir reportes y ocultar contenido falso o abusivo sin borrar el historial

5.4 Camino feliz de punta a punta

#	Paso	Módulo	Funciona en producción
1	El cliente se registra	M1 (Diego)	Sprint 1 · dom 15 nov
2	Publica una solicitud	M3 (Wilson)	Formulario dom 22 nov; módulo completo dom 6 dic
3	El proveedor la ve y envía una cotización	M2 (Daniel) + M4 (Diego)	Sprint 3 · dom 29 nov
4	El cliente compara y elige	M4 (Diego)	**Sprint 4 · dom 6 dic (núcleo demostrable)**
5	Se comunican por mensajes	M5 (Wilson)	Enviar y ver: vie 18 dic; terminado: vie 15 ene (MVP v1)
6	El cliente deja una reseña	M6 (Iker)	Formulario: vie 18 dic; mínimo: vie 15 ene; completo: vie 22 ene

Mientras M5 no exista, los pasos 4 y 5 se conectan con el campo de contacto del negocio (teléfono o WhatsApp). El flujo funciona desde el 6 de diciembre aunque todavía no haya chat.

5.5 User Story Map

Columnas: **Núcleo** (hasta el 6 dic) · **MVP v1** (hasta el 15 ene) · **Completo** (enero) · **Después** (no entra en el MVP).

Actividad	Núcleo	MVP v1	Completo	Después
Entrar a Offero	Ver la portada · registrarse con rol · iniciar y cerrar sesión · editar el nombre	—	—	Iniciar sesión con Google · recuperar contraseña (requiere SMTP)
Montar el negocio (proveedor)	Crear y editar el negocio · ver el listado y filtrar por categoría · ver el detalle	—	Logo del negocio (si el PO lo aprueba)	Varios negocios por proveedor · mapa
Pedir (cliente)	Publicar · ver mis solicitudes · editar y cancelar · los proveedores ven las abiertas	—	—	Fotos adjuntas · solicitud privada a un solo proveedor
Cotizar (proveedor)	Enviar una cotización · ver mis cotizaciones · retirarla	—	—	Contraofertas o negociación
Elegir (cliente)	Comparar y ordenar por precio o plazo · elegir una (atómico)	—	—	Pagos (excluidos por el documento)
Hablar	Contacto del negocio	Mensajes por solicitud · bandeja · marcar como leídos	Contador de no leídos · tiempo real (opcional)	Archivos en el chat · chats de grupo
Calificar (cliente)	—	Reseña de 1 a 5 con comentario · promedio	Editar durante 7 días · ordenar la lista	Respuesta del negocio
Moderar (admin)	—	Panel protegido · categorías · usuarios · reportes · ocultar	Cambiar roles · suspender cuentas	Métricas · historial de moderación
Seguridad (todas)	RLS en cada tabla · rutas protegidas · validación en el servidor	Revisión de RLS de las tablas nuevas	Revisión final de seguridad	—

5.6 Priorización MoSCoW

MoSCoW clasifica cada funcionalidad en Must (debe), Should (debería), Could (podría) y Won't (no en este MVP).

Prioridad	Funcionalidades	Justificación
Must	M0 completo · M1 completo · M2 completo · M3 completo · M4 completo · RLS, rutas protegidas y validación en el servidor en todo	Es el camino feliz (pasos 1 a 4). Sin esto no hay producto que validar (H1 a H3). La seguridad no es negociable (restricción 4)
Should	M5 mínimo · M6 mínimo · M7 mínimo (los 8 módulos en el MVP v1)	Completan los pasos 5 y 6 y permiten moderar con usuarios reales. Validan H4. El documento los incluye en el MVP
Could	M5 completo (contador; tiempo real opcional) · M6 completo (editar, ordenar) · M7 completo (roles, suspender) · extras: logo del negocio, búsqueda por nombre, Google	Mejoran la experiencia, pero el flujo funciona sin ellos. Planificados para enero, con margen
Won't (este MVP)	Pagos · app móvil · microservicios · IA avanzada · varios países (exclusiones del documento) · contraofertas · archivos adjuntos · chats de grupo · respuesta del negocio a reseñas	Fuera de alcance por el documento, o no aportan a validar las hipótesis

Orden de recorte si vamos atrasados (el detalle va en la Parte 9):

1. Extras de enero (logo, búsqueda, Google).
2. Tiempo real de M5.
3. Suspender cuentas y cambiar roles (M7 completo).
4. Editar y ordenar reseñas (M6 completo).
5. Contador de no leídos (M5 completo).
6. Solo en una emergencia antes del 6 de diciembre: M7 mínimo se reduce a categorías y ocultar. Los reportes se revisan en el panel de Supabase.

Nunca se recorta un Must ni la seguridad.

5.7 MVP mínimo demostrable

El recorte más pequeño que ya sirve para mostrarlo a usuarios reales es **el núcleo del 6 de diciembre**:

> Un cliente se registra y publica una solicitud. Un proveedor con negocio la ve y le envía una cotización. El cliente compara y elige, y contacta al proveedor con los datos de contacto del negocio. Todo corre en Vercel, con las empresas del piloto y sin datos inventados.

Si en una emergencia hubiera que recortar todavía más, lo único que podría esperar dentro del núcleo es **el listado público de negocios con filtro (M2)**. Crear el negocio sí es necesario, porque las cotizaciones salen de un negocio.